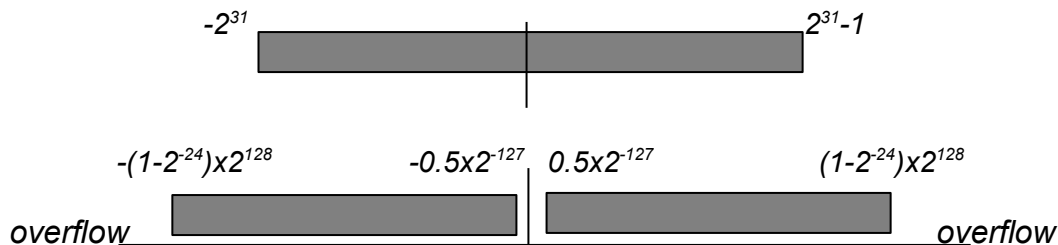


Notas de Aula – Guilherme Sipahi – Arquitetura de Computadores

Aritmética de Ponto Flutuante.

1. Da aritmética de Inteiros a aritmética de Pontos Flutuantes :

Números inteiros deixam de representar uma parte importante dos números reais: números fracionários, números irracionais, etc...



Ao observar a notação científica,

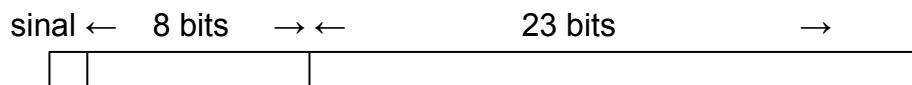
$$0.000000000000456 = 4,56 \times 10^{-13}$$

$$4560000000000 = 4,56 \times 10^{13}$$

se escrevermos na forma:

$$\pm S \times B^{\pm E}$$

podemos notar que a base (B) não precisa ser guardada.



Exemplos:

$$\begin{array}{ll} 0.11010001 \times 2^{10100} = 0 \ 10010011 & 1010 \ 0010 \ 0000 \ 0000 \ 0000 \ 000 \\ -0.11010001 \times 2^{10100} = 1 \ 10010011 & 1010 \ 0010 \ 0000 \ 0000 \ 0000 \ 000 \\ 0.11010001 \times 2^{-10100} = 0 \ 01101011 & 1010 \ 0010 \ 0000 \ 0000 \ 0000 \ 000 \\ -0.11010001 \times 2^{-10100} = 0 \ 01101011 & 1010 \ 0010 \ 0000 \ 0000 \ 0000 \ 000 \end{array}$$

Voltando à notação científica:

$$0.11010001 \times 2^{19} = 1.101001 \times 2^{18} = 0.011010001 \times 2^{20} \text{ (número normalizado)}$$

$$\pm 0.1bbb \times 2^{\pm E} \rightarrow b = 0 \text{ ou } 1.$$

Como o primeiro bit é sempre 1, pode-se excluí-lo da representação e utilizar 1 bit a mais, ficando com uma mantissa de 24 bits, dos quais 23 bits armazenados.

Porém os números armazenados na mantissa encontram-se na faixa de $0.5 < x < 1.0$

- O sinal é armazenado no primeiro bit;
- O primeiro bit da mantissa é sempre 1 e não precisa ser armazenado;
- A base do expoente é 2;
- Ao expoente armazenado é adicionado o valor 127.

$$0.5 \times 2^{-127} < 1 \times K (1-2^{-24}) \times 2^{128}$$

Problema: Não há a representação de zero!

2. Comparação das representações de inteiros e reais com 32 bits

32 bits $\rightarrow 2^{32}$ números para representar.

Inteiros $\rightarrow 2^{32}$ números igualmente espaçados;

ponto flutuante $\rightarrow 2^{32}$ espaçados irregularmente $\rightarrow$ o espaçamento dobra a cada vez que é incrementado o expoente.

3. Detalhes da representação de ponto flutuante

- Para aumentar a precisão é necessário aumentar o número de bits usados para descrever o número.
- Caso você mude a base, pode representar uma faixa maior de números, mas o espaçamento entre os números aumenta. Portanto, têm-se uma faixa maior de números representada com menor precisão.

4. Padrão IEEE 754 para a representação binária de ponto flutuante

Facilita a portabilidade de programas de um computador para outro.

- 3 formatos: **single precision** (32 bits, 8 bit expoente) - Base 2;
double precision (64 bits, 11 bit de expoente) - Base 2.
quadruple precision (128 bits, 15 bit de expoente) - Base 2.

Classes de números representados:

a) expoente: 00000001 $\rightarrow$ 11111110 : 1 - 254 $\rightarrow$ -126 $\Rightarrow$ 127
000000000001 $\rightarrow$ 111111111110 : 1 - 2046 $\rightarrow$ -1022 $\Rightarrow$ 1023
0000000000000001 $\rightarrow$ 1111111111111110 : 1-32768 $\rightarrow$ -16382 $\Rightarrow$ 16383
mantissa: 23 bits representados $\rightarrow$ número de 24 bits.
52 bits representados $\rightarrow$ número de 53 bits.
112 bits representados $\rightarrow$ número de 113 bits.

b) expoente: 00000000 mantissa: 23 0s $\Rightarrow$ zero; depende do sinal:
expoente: 000000000000 mantissa: 52 0s $\Rightarrow$ positivo/negativo.
expoente: 0000000000000000 mantissa: 112 0s

c) expoente: 11111111 mantissa: 23 0s $\Rightarrow$ dependendo do sinal:
expoente: 111111111111 mantissa 53 0s $\Rightarrow$ $-\infty$ ou $+\infty$
expoente: 1111111111111111 mantissa 113 0s
O usuário pode escolher se trata o overflow como erro ou continua o programa.

d) expoente: 00000000 mantissa: não-zero $\Rightarrow$ bit à esquerda
expoente: 000000000000 mantissa: não-zero $\Rightarrow$ 24°, 53° ou 113° bit =0
expoente: -126, -1022 ou -16382

e) expoente: 11111111 mantissa: não-zero $\Rightarrow$ Not a Number(NaN)
expoente: 111111111111 mantissa: não-zero $\Rightarrow$ exceções
expoente: 1111111111111111 mantissa: não-zero

Além dos formatos de simples e dupla precisão, o padrão define dois formatos estendidos, uma para precisão simples e outro para dupla.

- São usados para cálculos intermediários.
- Tem um número maior de bits para a mantissa e também para o expoente.
- Para maior precisão, diminui a chance do resultado ter sido contaminado por esse erro excessivo na aproximação.
- Por sua maior faixa de valores, reduz a chance de um overflow intermediário abortar uma computação cujo resultado poderia ser representado no formato básico.

5. Aritmética de ponto flutuante

Adição e subtração: é necessário que os dois operandos tenham o mesmo valor no expoente. Pode ser necessário mudar o ponto de um dos operandos para alinhar. Problemas que podem ocorrer com essas operações:

- *Overflow do expoente*: expoente excede o valor máximo do expoente. Em alguns sistemas pode ser designado $-\infty$ ou $+\infty$.
- *Underflow do expoente*: expoente negativo é maior que o número possível. É muito pequeno para ser representado e pode ser referido como zero.
- *Underflow da mantissa*: no alinhamento dígitos podem ser movidos para além do bit direito. Arredondamentos podem ser necessários.
- *Overflow da mantissa*: a adição de suas mantissas de mesmo sinal pode fazer com que o bit mais significativo esteja à esquerda do bit mais a esquerda. Isso pode ser corrigido pelo realinhamento.

a) 4 Passos para a adição e subtração:

1. Procurar por zeros;
2. Alinhar os operandos;
3. Somar ou subtrair os operandos;
4. Normalizar o resultado.

b) Operações Aritméticas com Floating Point:

$$X = X_s \times B^{X_e}$$

$$Y = Y_s \times B^{Y_e}$$

$$X + Y = (X_s \times B^{(X_e - Y_e)} + Y_s) \times B^{Y_e} \quad X_e \leq Y_e$$

$$X - Y = (X_s \times B^{(X_e - Y_e)} - Y_s) \times B^{Y_e}$$

$$X \times Y = (X_s \times Y_s) \times B^{(X_e + Y_e)}$$

$$X / Y = (X_s / Y_s) \times B^{(X_e - Y_e)}$$

c) Precisão

Problema: O alinhamento dos operadores causa perda de bits em um dos operandos. *Saída*: ter registradores extra para armazenar estes bits e reduzir erro com posteriores normalizações.

d) Arredondamento do Padrão do IEEE 754 (4 formas são permitidas):

i) Arredondamento para o mais próximo:

Caso o número esteja exatamente no meio, o arredondamento se fará para retornar o número par:

- se o último dígito for 1, arredonda para zero.
- se o último dígito for 0 , não arredonda.

ii) *Arredondamento para mais e menos infinito:*

- Usado para implementar aritmética de intervalos.

iii) *Truncagem.*

d) Infinito e NaN na aritmética de IEE 754

∞)

$$\begin{array}{ll} x + (+\infty) = +\infty & x / (+\infty) = 0 \\ x - (+\infty) = -\infty & (+\infty) + (+\infty) = +\infty \\ x + (-\infty) = -\infty & (-\infty) + (-\infty) = -\infty \\ x - (-\infty) = +\infty & (-\infty) + (+\infty) = -\infty \\ x \cdot (+\infty) = +\infty & (+\infty) + (-\infty) = +\infty \end{array}$$

NaN)

2 tipos:

- *sinalizada*: indicado uma operação em que um dos operandos é NaN. Aparece para valores não inicializados e números ou "melhorias aritméticas não pertencentes ao padrão."
- *não sinalizada*: não indica exceção e é carregada até o fim da operação.

Normas de definição para NaN:

Somas:

$$\begin{array}{l} (+\infty) + (+\infty) ==> \text{NaN} \\ (-\infty) + (+\infty) ==> \text{NaN} \\ (+\infty) - (+\infty) ==> \text{NaN} \\ (-\infty) - (-\infty) ==> \text{NaN} \end{array}$$

Multiplicação:

$$0 \times \infty ==> \text{NaN}$$

Divisão:

$$\begin{array}{l} 0/0 ==> \text{NaN} \\ \infty/\infty ==> \text{NaN} \end{array}$$

Restos de Divisão:

$$\begin{array}{l} x \text{ REM } 0 ==> \text{NaN} \\ \infty \text{ REM } 0 ==> \text{NaN} \end{array}$$

Raízes:

$$\sqrt{x}; x < 0 ==> \text{NaN}$$

[1] William Stallings - Computer Architecture and Organization, Pearson, 5th Edition